HELIXOR SOLVER EXAMPLE · GUIDE

Ocean container routing, explained

An importer has 232 containers to move from three Chinese ports to two US ports over six weeks, on four carriers. This guide walks through the business problems that planning raises, shows each one as a scenario, and explains how the hosted Helixor Solver handles it. Every number is measured, and you can reproduce it from this repository.

Repository: github.com/HelixorAI/helixor-solver-ocean-container-routing

---

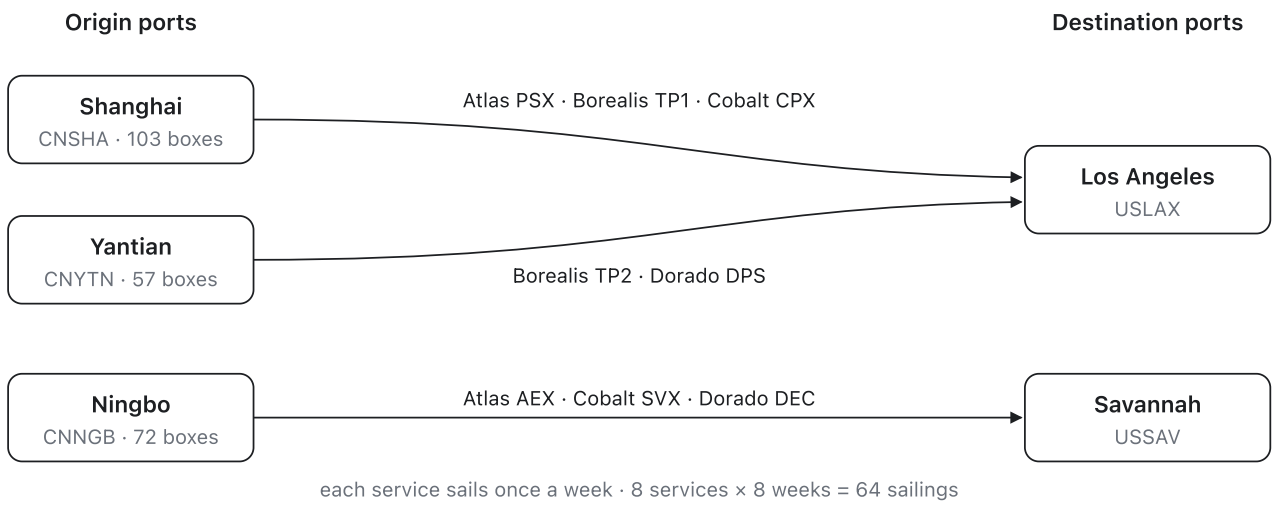
Contents

1. The business problem
2. What a good plan means
3. How the problem is modelled
4. How a plan is made and trusted
5. Scenario: the first plan
6. Scenario: taking over another system's plan
7. Scenario: the week goes wrong
8. Scenario: deciding what may change
9. Scenario: asking carriers for more space
10. Scenario: a peak-season surge
11. How good are the plans?
12. Calling the Solver API
13. A typed client, generated from the contract: SDK vs raw REST
14. Running the solver locally: the embedded runtime
15. Run it yourself
16. Glossary

---

1. The business problem

An importer buys goods from vendors in China and ships them to the US in containers. It doesn't own ships. It books space on **sailings**: one voyage of one carrier's service, leaving on a fixed day each week. In this example, four carriers run eight weekly services on three **lanes** (a lane is a port pair).



The network. Each arrow is a lane, labelled with the carrier services that sail it. A container can only sail on its own lane. The carriers and vendors are invented; the ports are real.

Four things make the booking hard:

Contracted space

Each carrier grants a weekly **allocation** on each sailing, for example 6 FEU. The importer has committed to it, so space it doesn't fill is money lost.

Spot space

Beyond the allocation a carrier may sell extra **spot** space at a higher rate, but only up to a limit, and only if it agrees.

Promise dates

Each container has a date its customer was promised. A sailing that arrives after it makes the container late.

Constant change

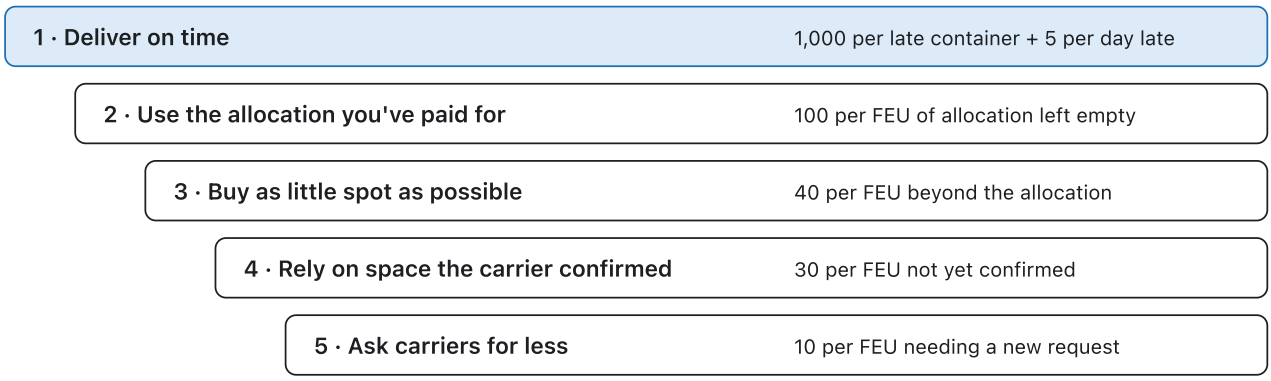
Sailings are blanked or delayed, cargo is late, vendors add orders, carriers cut allocations or refuse extra space. The plan has to be repaired without re-booking everything.

Sizes are counted in **FEU** (forty-foot equivalent units): a 40-foot container is 1 FEU, a 20-foot container is 0.5 FEU. The example has 155 40' high-cubes, 39 40' standard containers and 38 20' containers, and 224 FEU of allocation on the sailings of weeks 0 to 6.

---

2. What a good plan means

The business ranks its goals. The solver needs them as prices in one currency, so each goal becomes a weight per unit of shortfall. A higher goal gets a weight large enough that no amount of a lower goal can outweigh it in practice.



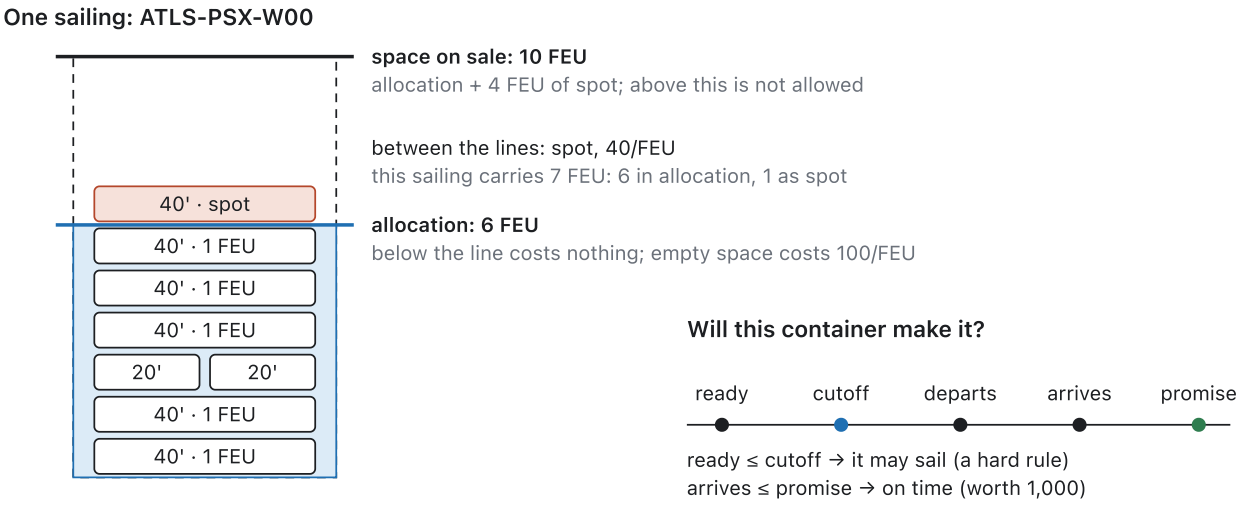
The objective. Lower in the list means cheaper, so the solver gives it up first. The numbers are the weights in the pack's manifest; a test checks that the independent scorer uses the same ones.

Why 100 for unused allocation and only 40 for spot? Allocation is paid for whether used or not, so leaving it empty and buying spot elsewhere is the worst of both. If your contracts work the other way, swap them; the weights are configuration, not code.

---

3. How the problem is modelled

The decision is simple to state: **for each container, pick one sailing**. It is bin packing (sailings are bins of limited space) combined with scheduling (each bin leaves and arrives on a fixed day).

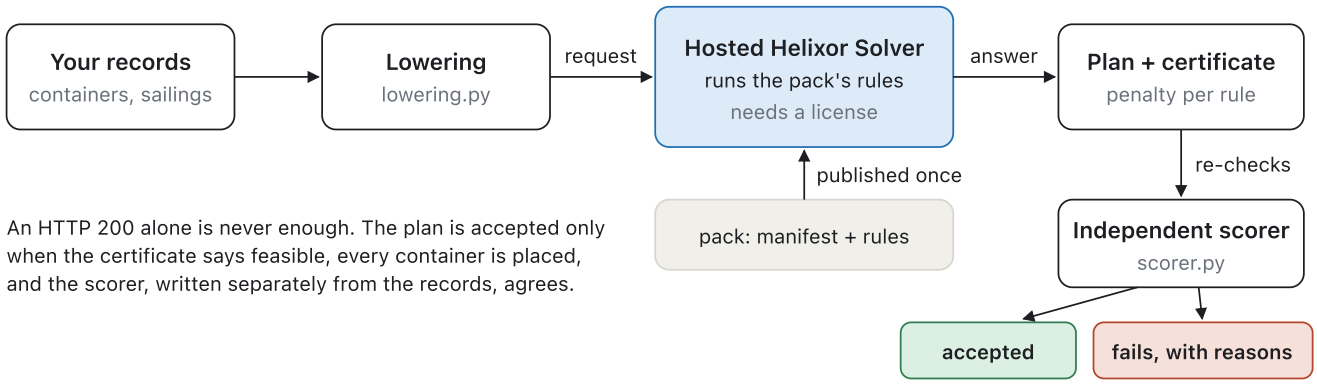


Left: a sailing is a bin with two lines. Filling up to the allocation is free; leaving it empty is the second-most expensive thing a plan can do. Space above the allocation is spot, up to the limit on sale. Right: a container can use a sailing only if its cargo is ready by the sailing's cutoff, and it is on time only if the sailing arrives by its promise date.

All of this is declared, not programmed. The **solver pack** (`pack/`) is a zip of configuration: a manifest naming the decision (containers onto sailings), eight rules, and their weights. Each rule uses a generic contract, such as "an attribute of the container equals an attribute of the sailing" for the lane rule, or "the

sum of FEU on a sailing stays below its capacity" for space. The solver has no ocean-shipping code. Your side does the **lowering**: it turns your records into the request, for example by working out, for each container, which sailings arrive on time.

4. How a plan is made and trusted



The trust path. The **certificate** is the solver's own account of the plan: feasible or not, and the penalty of each rule. The **independent scorer** reimplements the business rules directly from your records without the solver. If they disagree, the command stops; it never passes on a plan it couldn't verify.

5. Scenario: the first plan

> **BUSINESS**
> It's Monday of week 0. All 232 containers need a sailing for the next six weeks.

> **SOLVER**
> One request with every container and sailing, no prior plan. It searches for the cheapest assignment that breaks no hard rule.

> **YOU SEE**
> A feasible plan: 193 of 232 on time, 92% of the allocation used, 6.5–7.5 FEU of spot.

Engine	Penalty	Late (days)	Allocation used	Spot FEU	Time
Exact MIP (proven optimum)	42,740	39 (294)	92.2%	6.5	0.2 s
Helixor solver (next release)	42,920	39 (294)	91.7%	7.5	4.5 s

Both deliver the same 193 containers on time, the top goal. The Helixor plan is 0.4% above the proven optimum on total penalty. Why are 39 containers late in every plan? No sailing on their lane arrives before their promise date, so no plan can save them; the solver still chooses the sailing that makes them least late.

6. Scenario: taking over another system's plan

BUSINESS

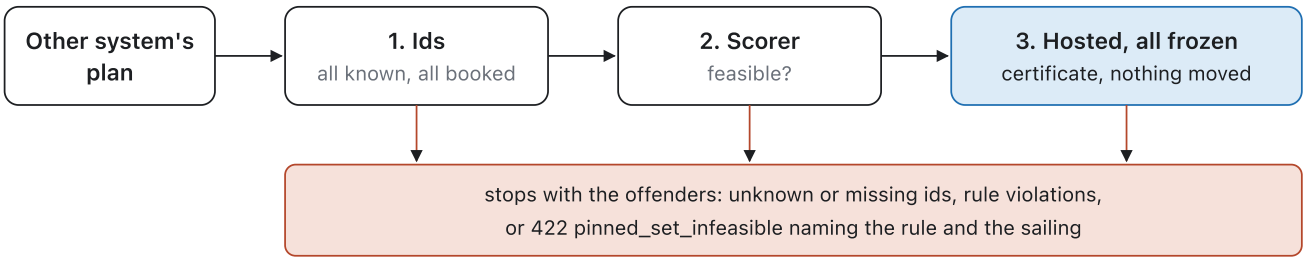
The importer already plans with another tool. It wants Helixor to take over from that plan, not start again.

SOLVER

The plan goes in with every booking *frozen*. The solver judges it under the pack's rules and moves nothing.

YOU SEE

The plan certified at the same penalty the scorer gives (42,780), or a refusal that names the broken rule and sailing.

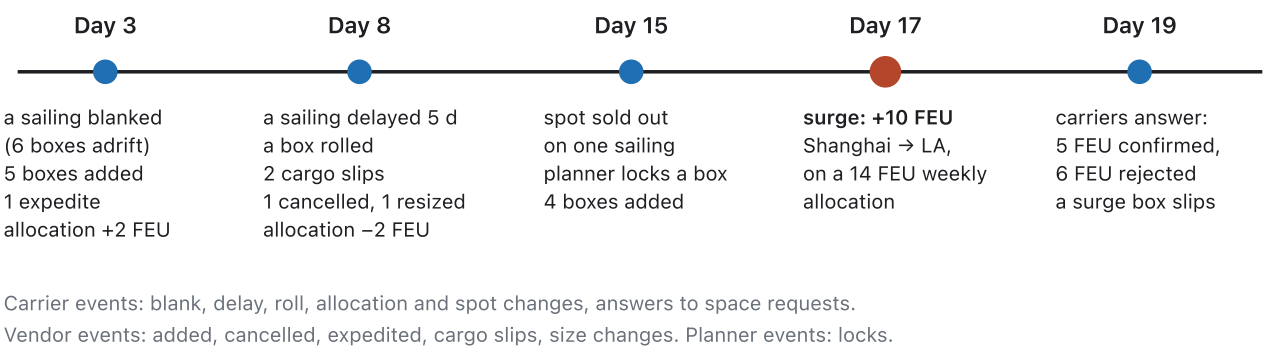


Adoption never quietly fixes a broken plan. If the frozen bookings alone break a hard rule (wrong lane, cargo not ready, a sailing over capacity), the solver refuses and says which. Once adopted, the plan is the starting point for everything that follows.

Bring the plan as `{"bookings": [{"container": …, "sailing": …}]}` or `{"placements": [{"entity": …, "group": …}]}`. The example adopts the exact MIP's plan, standing in for the other system.

7. Scenario: the week goes wrong

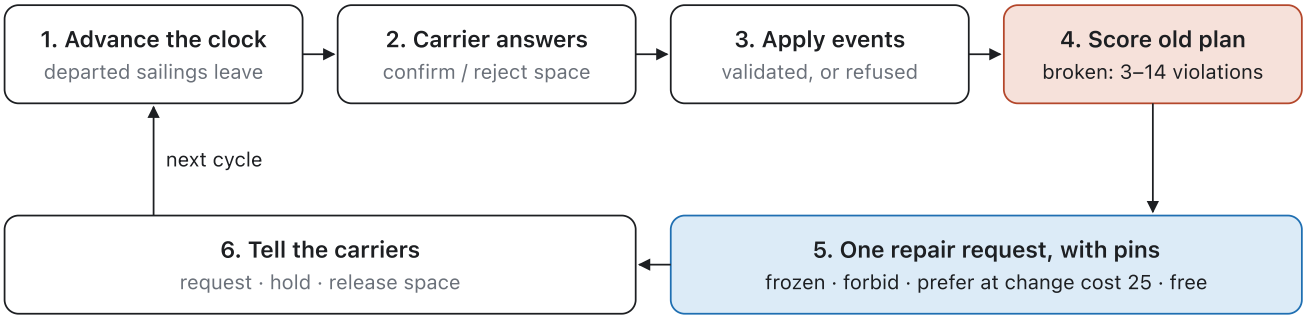
Plans don't survive the week. Every few days the planner rolls the clock forward, applies what happened, and asks for a **repair**: a new plan that fixes what broke and moves as little else as possible. The example replays five cycles over the first three weeks.



Carrier events: blank, delay, roll, allocation and spot changes, answers to space requests.
Vendor events: added, cancelled, expedited, cargo slips, size changes. Planner events: locks.

The replay in `data/events/replay.json`. Each event is validated; one that names an unknown sailing or container is refused, not skipped.

One cycle, step by step



Every repair in the replay comes back feasible, and the independent scorer confirms it.

The loop the `continuous` command runs. The old plan is always broken when an event lands; the repair makes it valid again.

Which containers may move?

Moving a container means re-booking it with a carrier, so the repair request carries a **pin** for every container. The replan policy chooses them:

Container	Pin	Why
Locked by a planner	`frozen`	an explicit decision
Sailing's cutoff within 7 days, in confirmed space	`frozen`	already booked with the carrier
Rolled off a sailing	`forbid` that sailing	the carrier won't take it
Touched by an event (blank, delay, slip, expedite, new)	none (free)	must be re-planned
Everything else	`prefer`, change cost 25	stays unless moving gains more than 25

A change cost of 25 means: move a container if that saves a late delivery (1,000) or fills a FEU of empty allocation (100), but not to shave four days off a container that is late anyway (4 × 5 = 20).

8. Scenario: deciding what may change

BUSINESS

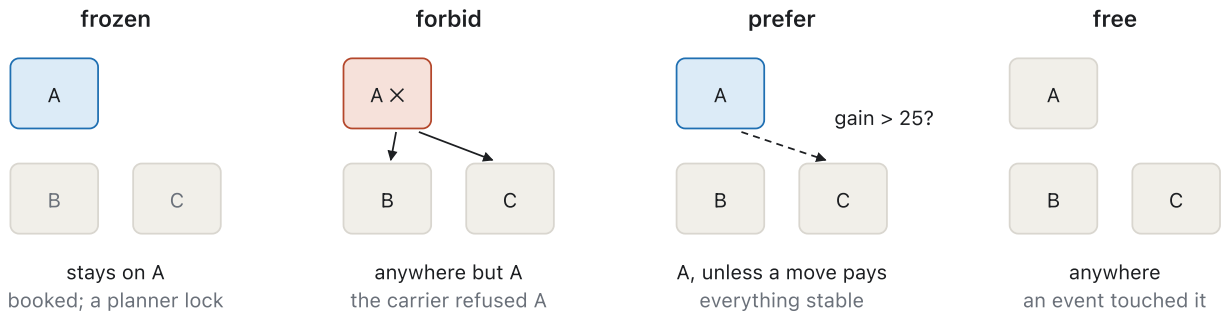
A carrier blanks one sailing (BORS-TP1-W01), and 5 containers lose their booking. How much of the rest of the plan should move?

SOLVER

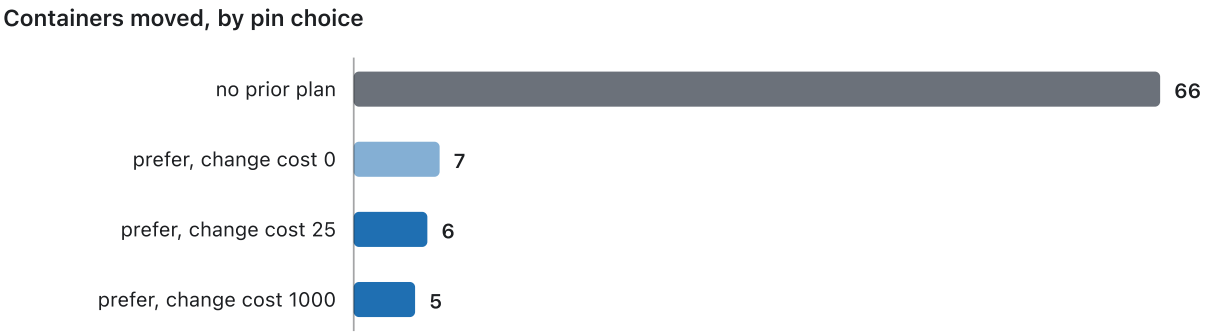
The same repair, run once per pin choice. Every pin is checked against the plan that comes back.

YOU SEE

The change cost works as a churn dial, and impossible pins are refused before any search.



The four pin modes for a container planned on sailing A. Hard rules still apply in every mode: a container can never go to a sailing on another lane or after its cutoff.



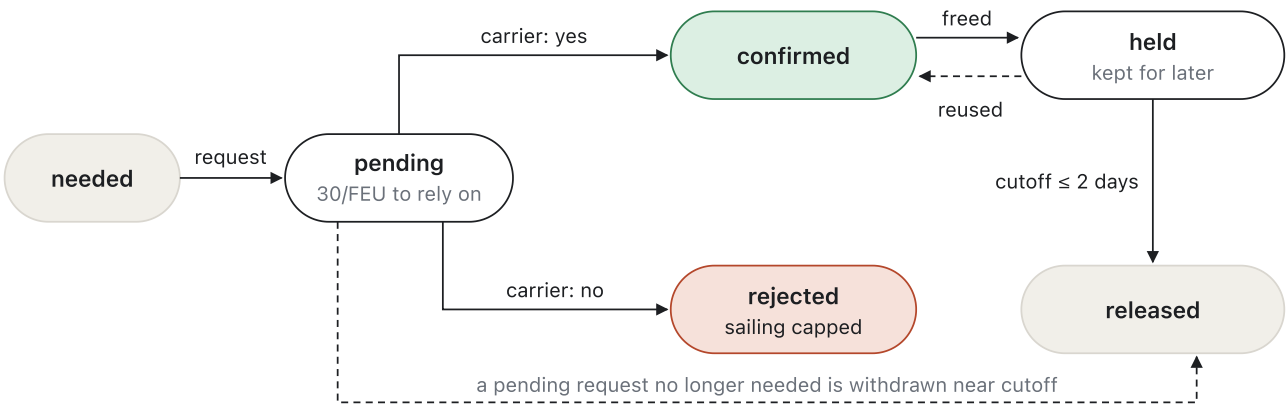
At 1000 only the 5 containers the blank displaced move, and the plan matches the optimum for those pins (44,115).

The change cost is the churn dial. Re-planning from scratch moves 66 containers. Keeping the prior plan as a preference moves 6–7 and scores better (44,025); at 1000 only the 5 that had to move do, for 90 more penalty.

Pins	Outcome	Penalty	Moved	Optimum, same pins
no prior plan	feasible	44,105	66	43,985
all prefer, change cost 0	feasible	44,025	7	43,985
all prefer, change cost 25	feasible	44,025	6	44,025
all prefer, change cost 1000	feasible	44,115	5	44,115
19 week-0 bookings frozen	feasible, pins held	44,045	6	44,045
3 containers forbidden COBT-CPX-W01	feasible, pins held	44,045	10	44,025
11 FEU frozen onto ATLS-PSX-W01 (10 FEU on sale)	**refused**: 422 `pinned_set_infeasible`, sailing_capacity in ATLS-PSX-W01	–	–	–

9. Scenario: asking carriers for more space

The allocation is a contract, so the carrier has already agreed to it. Anything beyond it is a request. The importer can **over-allocate**, asking for more than the allocation and seeing if the carrier accepts, often through the carrier's booking website. The carrier may say no. And when space is freed up, a planner doesn't necessarily hand it back, because a later change may need it.



How each FEU beyond the allocation moves through its life. The solver prices the uncertain states (30 per FEU not yet confirmed, 10 more per FEU needing a new request), so it prefers held space, then space already asked for, then a new request. It still goes beyond the allocation whenever that delivers a container on time.

BUSINESS

A booking on unconfirmed space isn't really booked. If the carrier says no, those containers must move.

SOLVER

Each sailing carries `confirmed_feu` and `committed_feu` (confirmed plus pending). After every plan, the planner works out requests, holds and releases. A rejection lowers the sailing's capacity, and the next repair moves what no longer fits.

YOU SEE

Lines like `carrier action: request 2 FEU on ATLS-PSX-W05` and `BORS-TP1-W03: carrier REJECTED 1 FEU; space now capped at 5 FEU`.

Two details follow from the business rule. The replan policy freezes a booking near cutoff only within confirmed space, because a booking on pending space isn't with the carrier yet; the run says so in a note. And in the replay, a simulator plays the carriers: the scenario's hidden `simulation` block says how much extra each carrier will really take. The planner never reads it and it is never sent to the solver; a test checks this. Plans are made on what carriers have said, not on what they will say.

In the replay, after cycle 1 the plan holds 0.5 FEU of confirmed space it no longer needs on BORS-TP2-W02. It keeps it through cycle 2 in case a later change needs it, and releases it in cycle 3, once that sailing's cutoff is within two days.

---

10. Scenario: a peak-season surge

BUSINESS

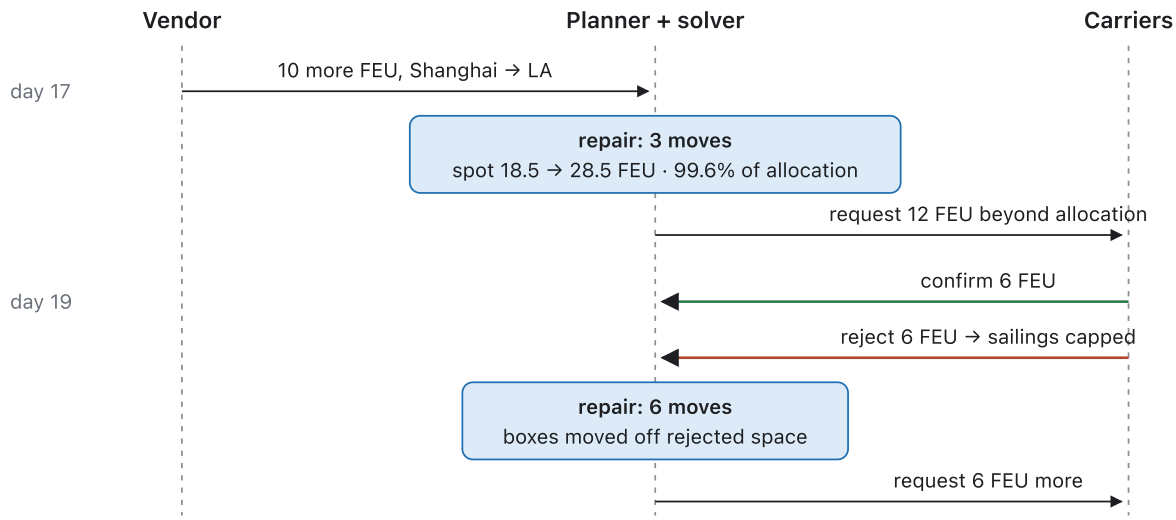
On day 17 a vendor adds 10 FEU for Shanghai to Los Angeles. The lane's weekly allocation is 14 FEU and already nearly full. The demand can't fit in the contract.

SOLVER

It fills the allocation and buys spot for the rest, choosing sailings that keep the most containers on time. The planner requests the extra space.

YOU SEE

Spot rises from 18.5 to 28.5 FEU with 99.6% of the allocation used; then carriers reject 6 of 12 FEU, and the next repair re-routes.



Over-demand that needs spot, and what happens when carriers say no. The rejected FEU become hard capacity limits; the containers on them break the plan, and the next repair moves them to space that exists.

11. How good are the plans?

An **exact MIP** gets the exact same request as the Helixor solver and proves the optimum. A from-scratch hosted solve shows what re-planning everything would move.

Cycle	Helixor repair	Optimum, same pins	From scratch
1 · day 3	45,160 · 16 moved	45,160 · 13	45,220 · 72
2 · day 8	50,040 · 14	50,005 · 13	48,790 · 72
3 · day 15	46,825 · 5	46,825 · 2	43,705 · 60
4 · day 17 (surge)	54,825 · 3	54,825 · 3	46,890 · 54
5 · day 19	53,740 · 6	53,740 · 6	45,815 · 77

Penalty excludes the change cost; "moved" counts containers whose sailing changed from the previous plan.

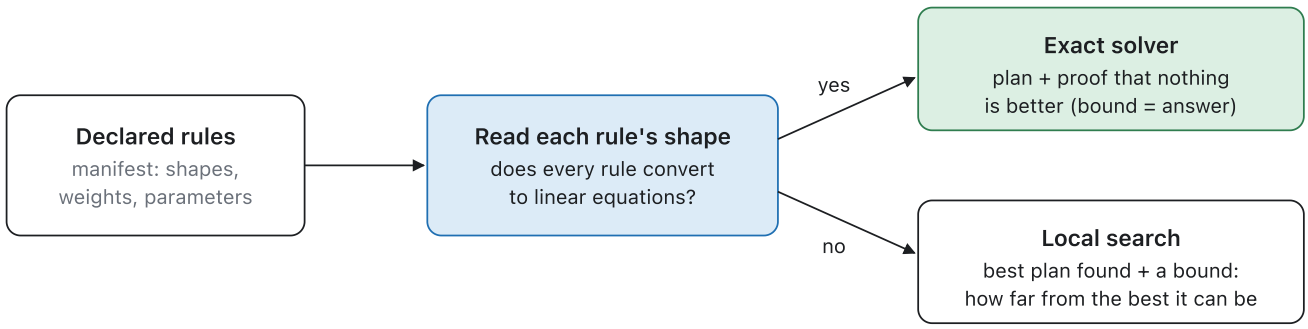
- **Every repair is feasible**, confirmed by the independent scorer.
- **The Helixor repair reaches the proven same-pins optimum in cycles 1, 3, 4 and 5**, and is 0.07% above it in cycle 2 (35 points), in 2.1–4.1 seconds.
- **Repair moves far less than re-planning:** 3–16 containers against 54–77. Where it moves a few more than the optimum, the extra moves are free containers (ones an event touched), which carry no change cost, so they leave the score unchanged.
- **From scratch scores lower from cycle 2 on** because nothing holds it to the bookings already with carriers. It reaches that by moving 54–77 containers, frozen bookings included, which is why it isn't a plan anyone would send.

Measured on the next solver release. These figures come from the next Helixor solver release, run on a local server with the same requests; the hosted service gets it on its next deploy. That release closed a gap the hosted repair had in these cycles (4.5–10% above the same-pins optimum) with two generic search changes: each scoring level now judges a worse move in its own units, and every solve ends with a pass that proves no single move or swap improves the plan. Neither change knows about containers or sailings.

Why a declared model can be proven optimal

The exact MIP in this example (`ocean_routing/optimum.py`) is not a second model of the business. It restates the manifest's own rules as equations, rule for rule, and it proves the optimum of every cycle in 0.02–0.3 seconds. That is possible because each rule in the manifest has a declared shape, and every one of those shapes can be written as linear equations:

Rule	Declared shape	As equations
lane match, cargo ready by cutoff	which sailings a container may use	the container's choice is restricted to those sailings
sailing capacity	a sum over a sailing, at most a limit	one inequality per sailing
on-time delivery, days late	a cost for each container–sailing pair	a coefficient on that choice
allocation, spot, unconfirmed and new space	a cost of a sailing's load above or below a line	one extra variable and one inequality per line



one model: the same declared rules, weights and configuration drive both

What a declared model makes possible. Because the solver can read what each rule is, not only what it scores, it can choose a method that proves its answer when the rules allow it, and say how close to the best its answer is when they don't.

What follows from the model being declared rather than programmed:

- **A proven answer where the rules allow one.** When every rule converts, the answer can come with a proof that no better plan exists, not just a good plan.

- **A measured gap where they don't.** Converting the rules that do convert gives a bound, so a plan found by search can say how far from the best possible it can be.
- **One model to maintain.** The equations come from the manifest, so a change to a rule, a weight or a customer's configuration reaches the exact method and the search at once. Nobody keeps a second copy in step.
- **Every answer stays checkable.** Whichever method produced the plan, the certificate scores it against the same declared rules, and the independent scorer re-checks it.

> **Status.** In this example the MIP is written by hand from the manifest, as a yardstick. The current Solver server code also reads the manifest's rules and, when every rule converts, returns its plan with a proof: the demo-week solve in section 12 came back with `proven_optimal: true`. That has not been confirmed on the hosted service yet, so read `proven_optimal` in each answer rather than assume it. Rules about sequences and timing along a route, and very large instances, will stay with search.

---

12. Calling the Solver API

Every hosted command in this repository makes the same few REST calls, and `ocean_routing/client.py` is the shortest complete client. This section documents those calls for this problem type: what to send, what comes back, and how to act on it. Every request and response below was run against a Helixor Solver server with pack 2.4 published (sha256 `fe84dcc6…8e34418`), and the values shown are the ones it returned. That server ran locally with sign-in switched off, so `$SOLVER_TOKEN` stands for the token the hosted service checks.

Before the first solve: publish and select the pack

The model is not in the request. Publishing the pack makes its problem type, `ocean_container_routing`, available to your organization, and every solve of that type runs the **active** version of the pack. Nothing in the solve body picks a manifest. A request that carries its own `options.manifest` is refused by a default deployment (`422 unsupported_problem_type`), so a solve runs only rules that were validated and admitted with a pack.

Call	What it does
`POST /api/v1/solver-packs/validations?expected_sha256=…`	Validates the zip (body `application/zip`) without storing it. The demo pack answers `ADMITTED`, 0 errors, 0 warnings.
`POST /api/v1/solver-packs?activate=true&expected_sha256=…`	Stores the version and makes it active. A version is immutable. Publishing 2.4 again with different bytes is refused with `409 solver_pack_version_conflict`, which names both hashes. Earlier server releases answer `422 solver_pack_rejected`. Either way, any change needs a new version.
`GET /api/v1/solver-packs/ocean_container_routing/versions`	Every stored version, with `active`: the `pack_id`, `version` and `sha256` that solves run.
`GET /api/v1/solver-packs/ocean_container_routing/history`	The pack's append-only log: one event per change, with `kind` (`published`, `activated`, `superseded`, `retired` or `deleted`), `version`, `sha256`, `actor`, `at` and the version it replaced.
`GET /api/v1/manifests/ocean_container_routing`	The active manifest: rules, `data_schema`, pinning and `ui.visualizations`. Read the KPIs from here (see Using the answer).
`GET /api/v1/datasets/demo/scenarios/ocean_container_routing`	The pack's demo scenarios. This pack has one: `two_lane_week` ("Two lanes, one week").
`GET /api/v1/datasets/demo/problem/ocean_container_routing?scenario=two_lane_week`	That scenario's request payload under `data`, the same file as `pack/datasets/two_lane_week.demo.json`: 41 containers, 15 sailings.

The solve path never makes up data. A scenario is chosen only on the dataset route. To solve the demo week, you fetch it and send it as `problem`.

The solve request

```
curl -X POST "$SOLVER_URL/api/v1/solve/ocean_container_routing" \
  -H "Authorization: Bearer $SOLVER_TOKEN" \
  -H "Content-Type: application/json" \
  -H "X-Helixor-Solve-Id: demo-week-0001" \
  --data-binary @solve-demo.json
```

The path parameter is the pack's problem type. The headers:

Header		Meaning
`Authorization: Bearer <token>`	required	Your own Helixor token. Your organization and its license come from the token. Do not send `X-Realm` or `X-Tenant-Id`. Without a token the answer is `401` with `failure_code: invalid_token`.
`Content-Type: application/json`	required	
`X-Helixor-Solve-Id`	optional	A name you choose for this solve: 8–128 letters, digits, `-` or `_`. Any other value is refused with `422 invalid_solve_id`. While the solve runs, and for 10 minutes after it ends, `GET /api/v1/solve/runs/{id}` shows its `status`, `phase` and `job_id`. This is how a caller that lost the response finds the answer again. `POST /api/v1/solve/runs/{id}:stop` asks a running solve to stop.

The body has three objects:

```
{
  "problem": {
    "containers": [
      {"id": "CNSHA-0028", "lane": "CNSHA-USLAX", "size": "40HC", "feu": 1.0,
       "ready_day": 12, "promise_day": 36,
       "on_time_sailings": ["ATLS-PSX-W01", "ATLS-PSX-W02", "BORS-TP1-W01",
                            "BORS-TP1-W02", "COBT-CPX-W01"]},
      ...                                    // 41 containers
    ],
    "sailings": [
      {"id": "ATLS-PSX-W01", "carrier": "ATLS", "service": "PSX", "lane": "CNSHA-USLAX",
       "week": 1, "cutoff_day": 6, "etd_day": 8, "eta_day": 23,
       "allocation_feu": 6.0, "capacity_feu": 10.0,
       "confirmed_feu": 6.0, "committed_feu": 6.0},
      ...                                    // 15 sailings
    ]
  },
  "solver": {"mode": "local"},
  "options": {}
}
```

Field	Meaning
`problem.containers[]`	The entities. The manifest's `data_schema` requires `id`, `lane`, `feu`, `ready_day`, `promise_day` and `on_time_sailings` (the ids of the sailings on the container's lane that arrive by `promise_day`; your lowering derives this list). The rule views also read `size`.
`problem.sailings[]`	The groups. Required: `id`, `lane`, `cutoff_day`, `eta_day`, `allocation_feu`, `capacity_feu` (allocation plus the spot space on sale), `confirmed_feu` (space the carrier has acknowledged) and `committed_feu` (acknowledged plus requested). The views also read `carrier`, `week` and `etd_day`.
`problem.prior_plan`	Repair only: the previous plan, as `{"container id": "sailing id"}`.
`problem.pins[]`	Repair only: `{"entity": id, "mode": "frozen" | "forbid" | "prefer", "group": sailing id}`. `frozen` keeps the container on `group`. `forbid` keeps it off `group`. `prefer` keeps it on its prior sailing unless moving pays more than `change_cost`.
`problem.pin_default`	Repair only: the mode for every container that has no pin of its own, for example `"prefer"`.
`problem.change_cost`	Repair only: the price of moving a `prefer` container, in the same units as the rule weights (manifest default 25).
`solver.mode`	Execution mode. `"local"` runs the solve on the Solver service itself, and that is the mode this example uses. Asking for a GPU or cloud backend or device needs the `solver.gpu` entitlement. Without it the answer is `403 remote_solve_not_entitled`, with a `repair_hint` telling you to drop that option. Pinned solves always run on the service.
`options`	Send `{}` for this pack: the search budget comes from the manifest's `search:` block. `options.constraints` is an object of `{"rule id": {parameter overrides}}` (not a list). It can switch on a rule that a manifest leaves off and override a rule's parameters. It cannot switch a rule off: `{"minimize_spot": {"enabled": false}}` is accepted and ignored, and the penalty stays 1,550. Every rule in this pack is on by default. A list here currently fails with a plain `500`. Budget keys are checked against the service's ceilings: `time_limit_seconds: 9999` is refused with `422 input_limit_exceeded`, and `time_limit_seconds: 1` returned the same plan. Keys that start with `__` belong to the service and are refused with `422 reserved_option_key`.

A repair sends the same records plus the previous plan and the pins. This request froze one container, forbade a second container's sailing, and kept every other container where it was unless moving it paid:

```
"problem": {
  "containers": [...], "sailings": [...],
  "prior_plan": {"CNSHA-0028": "ATLS-PSX-W02", "CNSHA-0029": "ATLS-PSX-W02", ...},
  "pins": [{"entity": "CNSHA-0028", "mode": "frozen", "group": "ATLS-PSX-W02"},
           {"entity": "CNSHA-0029", "mode": "forbid", "group": "ATLS-PSX-W02"}],
  "pin_default": "prefer",
  "change_cost": 25.0
}
```

What comes back

The demo week, solved (an excerpt; `…` marks lists and diagnostics left out):

```
HTTP/1.1 200 OK
{
  "status": "completed",
  "job_id": "job-1cb4397b9ec1",
  "solver_used": "helixor-solver-server-local",
  "actual_provider": "local", "actual_device": "cpu", "solve_time_ms": 31,
  "solution": {
    "solved_by": "manifest",
    "placements": [{"entity": "CNSHA-0028", "group": "ATLS-PSX-W02"},
                   {"entity": "CNSHA-0029", "group": "ATLS-PSX-W02"}, …],
    "entity_ids": ["CNSHA-0028", "CNSHA-0029", "CNSHA-0030", …],
    "group_ids":  ["ATLS-PSX-W01", "ATLS-PSX-W02", "ATLS-PSX-W03", …],
    "assignment": [[], [0, 1, 5, 8, 9, 12, 13, 16, 19, 20], …],
    "groups_used": 9, "entities_placed": 41, "entities_total": 41,
    "is_feasible": true, "penalty": 1550.0, "objective": 0.0,
    "admission": {
      "is_feasible": true, "score": -1550.0,
      "eligibility_penalty": 0.0, "hard_penalty": 0.0,
      "soft_penalty": 1550.0, "unplaced_penalty": 0.0,
      "unplaced_entities": [], "unplaced_evaluated": true,
      "evaluated_constraints": ["lane_match", "cargo_ready_by_cutoff", "sailing_capacity",
        "on_time_delivery", "allocation_utilization", "minimize_spot", "days_late",
        "unconfirmed_space", "new_space_request"],
      "reason": "feasible with penalty: allocation_utilization (penalized, penalty 11.5 x weight
100); …",
      "violations": [
        {"name": "allocation_utilization", "is_hard": false, "penalty": 11.5,
         "weight": 100.0, "weighted_penalty": 1150.0, "affected_groups": [0, 7, 9]},
        {"name": "minimize_spot", "is_hard": false, "penalty": 5.0,
         "weight": 40.0, "weighted_penalty": 200.0, "affected_groups": [1, 2, 10]},
        …]
    },
    "exact": {"status": "optimal", "proven_optimal": true, …},
    "unevaluable_rules": []
  },
  "result": {
    "solution": {…}, "is_feasible": true, "admission": {…}, "manifest_driven": true,
    "metadata": {"solver_pack": {"pack_id": "ocean_container_routing", "version": "2.4",
      "sha256": "fe84dcc611140a630aa7506310449b296ac3be5b1e0f51568ba6db36d8e34418",
      "requested_problem_type": "ocean_container_routing"}}
  },
  "objective_value": 0.0,
  "solver_version": {"git_commit": "…", "build_time": "…", "image_tag": "…"}
}
```

Field	Meaning
`status`	`completed` means the solve finished. It does **not** mean the plan is feasible: an infeasible plan also comes back `200`, `completed` (see the failure table).
`job_id`	`GET /api/v1/solve/{job_id}` returns this same body to the caller that ran the solve, and `404` for an unknown id.
`solution.placements[]`	The plan: one `{entity, group}` per booked container, meaning container id and sailing id.
`solution.entity_ids`, `group_ids`, `assignment`	The same plan by index: `assignment[i]` lists the container indexes (into `entity_ids`) booked on sailing `group_ids[i]`. `groups_used`, `entities_placed` and `entities_total` count them.
`solution.is_feasible`, `admission.is_feasible`	The verdict under the pack's rules. Act only when it is `true`.
`solution.penalty`, `admission.score`	The plan's score: the total weighted penalty (1,550 here), and its negative. Lower penalty is better. The admission splits it into bands: `eligibility_penalty`, `hard_penalty`, `soft_penalty` and `unplaced_penalty`.
`objective`, `objective_value`	The cost of the manifest's declared `objectives`. This pack declares none and expresses everything as rules, so these are 0. Rank plans by `penalty`.
`admission.evaluated_constraints`	Every rule that judged the plan: all nine of the pack's rules.
`admission.violations[]`	One entry per rule that charged a penalty: `name`, `is_hard`, `penalty` (in the rule's own units: containers, FEU or days), `weight`, `weighted_penalty`, and `affected_groups` (indexes into `group_ids`). `reason` says the same in one sentence.
`admission.unplaced_entities`	Indexes (into `entity_ids`) of containers the plan could not book. It is empty in a usable plan.
`solution.exact.status`, `proven_optimal`	Whether the plan comes with a proof that no better plan exists under these rules: `"optimal"`, `true` for the demo week. When it is `false` the plan is still judged by the same rules, only without that proof. The other keys under `exact`, and the `improvement` block, are engine diagnostics, not part of the contract. Don't build on them.

Field	Meaning
`solution.unevaluable_rules`	Rules the engine could not evaluate on this request. It is empty here. Don't act on a plan where it is not.
`solution.pins`, `result.pins`	Repairs only: `requested`, `frozen` / `frozen_honored`, `forbid` / `forbid_honored`, `prefer` / `prefer_kept` / `prefer_moved`, `change_cost`, `changes_vs_prior`, `violations.frozen` and `violations.forbid` (both empty when every pin held), and `placement_provenance`, which marks each container `frozen`, `kept` or `moved`. In the repair above, 1 of 1 frozen and 1 of 1 forbid pins held, all 39 `prefer` containers stayed, and only the forbidden container moved (to COBT-CPX-W02). The penalty rose to 2,420, mainly because that container now arrives late (1,000 for the late container plus 50 for its 10 days).
`result.metadata.solver_pack`	Provenance: the `pack_id`, `version` and `sha256` of the pack whose rules produced this answer, and the problem type you asked for.
`solver_used`, `actual_provider`, `actual_device`, `solve_time_ms`, `solver_version`	Where and how long it ran (here on the service, on CPU) and the service's build stamp (`git_commit`, `build_time`, `image_tag`).

The rule results for the demo week, read from `evaluated_constraints` and `violations`, with each rule's type taken from the manifest:

Rule	Type	Penalty	Weight	Weighted	Sailings named
`lane_match`	eligibility	0		0	
`cargo_ready_by_cutoff`	eligibility	0		0	
`sailing_capacity`	hard	0		0	
`on_time_delivery`	soft	0	1,000	0	
`allocation_utilization`	soft	11.5 FEU	100	1,150	ATLS-PSX-W01, COBT-CPX-W02, BORS-TP2-W01
`minimize_spot`	soft	5 FEU	40	200	ATLS-PSX-W02, ATLS-PSX-W03, BORS-TP2-W02
`days_late`	soft	0	5	0	
`unconfirmed_space`	soft	5 FEU	30	150	the same three
`new_space_request`	soft	5 FEU	10	50	the same three
Total				**1,550**	feasible, proven optimal

A rule without a weight blocks the plan if it is broken. A soft rule only charges its penalty. The decision block adds one more hard rule that the manifest does not list: `total_assignment`, which says every container must be booked. It appears in `violations` when a plan leaves containers unbooked.

When the solve fails

The service fails closed. It refuses a request it cannot solve rather than answer something else, and a 422 fails the same way every time you repeat it. Failure bodies come in three shapes, so read the code from the right place:

- **Solve failures** (`missing_problem_data`, `input_limit_exceeded`, `reserved_option_key`, `unsupported_problem_type`, `remote_solve_not_entitled`, `solver_pack_version_conflict`) carry `failure_code` and `detail` at the top level. Input faults also carry `status: "unsupported"`, `problem_type`, `missing_inputs`, `error_class`, `http_status`, `job_id` and a `verdict` with `status: "failed"` and an `inputs_hash`.
- **Request and caller checks** (`invalid_solve_id`, `invalid_token`, and the run lookups) nest them one level down: `{"detail": {"failure_code": …, "detail": …}}`.
- **A body of the wrong shape**, such as `"solver": "local"`, is a 422 whose `detail` is a list of field errors with no `failure_code`. An unknown job is a 404 whose `detail` is a sentence.

One gap: `options.constraints` sent as a list currently returns a plain `500` instead of a typed refusal. Send it as an object.

HTTP	`failure_code`	Seen when	What to do
200	*none*: `is_feasible: false`	The demo week with no spot space on sale (`capacity_feu = allocation_feu`). 39 of 41 containers were placed, `unplaced_entities: [19, 27]` (CNSHA-0047, CNYTN-0072), and `total_assignment` was broken (hard, penalty 2). The reason was `"infeasible: total_assignment (blocked, …)"` .	Do not act on the placements. Change the inputs (more space, later promises) and solve again.
422	`missing_problem_data`	A container without `on_time_sailings` . `missing_inputs: ["containers[0].on_time_sailings"]` .	Fix the lowering. Retrying unchanged fails the same way.
422	`input_limit_exceeded`	`options.time_limit_seconds: 9999` ("exceeds the hosted ceiling of 600").	Stay under the ceiling, or leave the budget to the pack.
422	`reserved_option_key`	An option key starting with `__` .	Remove it.
422	`unsupported_problem_type`	A problem type with no active pack (the body lists `supported_problem_types`), or `options.manifest` in the request, which a default deployment refuses.	Publish and activate the pack, then solve its type.
422	`invalid_solve_id`	An `X-Helixor-Solve-Id` outside 8–128 letters, digits, `-` , `_` .	Use a valid name.
403	`remote_solve_not_entitled`	`solver.mode: "gpu"` on an account without `solver.gpu` . Names the `entitlement` and a `repair_hint` .	Send `"mode": "local"` .
401	`invalid_token`	No bearer token.	Send your token.
404		`GET /api/v1/solve/{job_id}` for an unknown job.	Use the `job_id` the solve returned, or look it up by solve id.

Using the answer

An HTTP 200 alone is never enough. The example's planner refuses an answer that is not feasible, that leaves a container unbooked, or that the independent scorer rejects. Against the raw response, a full acceptance check also covers the penalty bands, the rule evaluation and the pack's provenance:

```
body = response.json()                      # after checking response.status_code == 200
sol, adm = body["solution"], body["solution"]["admission"]
pack = body["result"]["metadata"]["solver_pack"]
accepted = (
    body["status"] == "completed"
    and sol["is_feasible"] and adm["is_feasible"]
    and adm["eligibility_penalty"] == 0 and adm["hard_penalty"] == 0
    and not adm["unplaced_entities"]
    and sol["entities_placed"] == sol["entities_total"]
    and not sol["unevaluable_rules"]
    and (pack["version"], pack["sha256"]) == ("2.4", PUBLISHED_SHA256)
)
if not accepted:
    raise SystemExit(adm["reason"])         # keep the previous plan; never pass this one on

by_container = {p["entity"]: p["group"] for p in sol["placements"]}
by_sailing = {}
for container, sailing in by_container.items():
    by_sailing.setdefault(sailing, []).append(container)
```

Then, from your own records:

- **Per container:** `by_container` is the booking to make. CNSHA-0028 goes on ATLS-PSX-W02. In a repair, `pins.placement_provenance` tells you which bookings are new (`moved`) and which stand.
- **Per sailing:** add up the FEU on each sailing and compare it with that sailing's `allocation_feu`, `confirmed_feu` and `capacity_feu`. FEU above the allocation is spot, and FEU above `committed_feu` is space still to request from the carrier. In the demo week, ATLS-PSX-W02 carries 9 FEU on a 6 FEU allocation, and ATLS-PSX-W03 and BORS-TP2-W02 carry 1 FEU over theirs: 5 FEU of spot in total, all still to request. ATLS-PSX-W01 and COBT-CPX-W02 carry nothing, and BORS-TP2-W01 carries 2.5 FEU of its 5.
- **Which sailings a rule names:** map `affected_groups` through `group_ids`. That turns `minimize_spot`'s `[1, 2, 10]` into the three sailings above, which are the carriers to ask for space.
- **KPIs and views:** the pack declares them in the manifest's `ui.visualizations` (`GET /api/v1/manifests/ocean_container_routing`), not in the solve response. Each KPI names a rule and a measure, and you evaluate it from your records and the placements, the way `ocean_routing/report.py` and the pack's solution surface do. For the demo week they come out as: on time 100%, late containers 0, allocation used 73.9% (32.5 of 44 FEU), spot space 5 FEU, days late 0, on unacknowledged space 5 FEU, space still to request 5 FEU. These agree with the certificate: 11.5 FEU of allocation unused, and 5 FEU on `minimize_spot`. The other declared views, load by sailing, allocation used by week and carrier, the sailing schedule and late containers, bind to the same records and placements.
- **Re-check it yourself:** `python -m ocean_routing score` scores any plan from the records alone, without the solver.

> **Never read KPIs off an infeasible plan.** In the no-spot week the views count the two unplaced containers as late (5 late, against 3 in `on_time_delivery`), and the plan cannot be booked anyway. Check feasibility first.

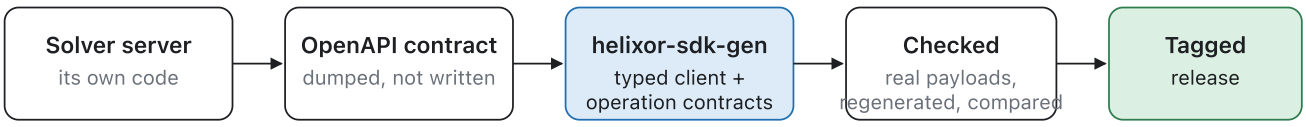
Handling failures. A 422 is a fault in the request: fix the input and send it again, because retrying unchanged repeats the failure. A 403 names the entitlement or option to drop. If the connection drops during a solve, don't start another solve straight away. Send every solve with an `X-Helixor-Solve-Id`, then read `GET /api/v1/solve/runs/{id}` for its `job_id` and fetch the answer with `GET /api/v1/solve/{job_id}`. Both are kept in the memory of the server instance that ran the solve. Behind several instances, or after a restart, the lookup can answer 404, and then you solve again. A stop request can answer `409 solve_not_stoppable` when the solve is running where a stop cannot reach it. Set your client timeout above the solve's budget: the generated client below defaults to 120 seconds. A repair should keep the previous plan until a new one passes the acceptance check.

---

13. A typed client, generated from the contract

This repository calls the API with plain HTTP, so it stays public and needs nothing but `httpx`. Helixor also provides typed clients for its APIs. They are generated by Helixor's own SDK generator, **helixor-sdk-gen**, from each service's OpenAPI contract. For the Solver, the client is the Python package `helixor-solver-contract`. It covers solve, job status, pre-solve checks, recommendations, health and version.

How the client is produced



a server change reaches the client only by regenerating it from the new contract

Nobody writes the client by hand. The Solver server produces its own OpenAPI contract. The generator reads that contract and emits the client plus `operation-contracts.json`, a language-neutral record of every operation with a hash of its exact wire shape. Tests parse payloads captured from the server application, and before a release the client is regenerated and compared with what is committed.

1. **The contract comes from the server.** The Solver server dumps the solve-contract part of its OpenAPI from the same models that validate requests. Before writing this section we dumped the contract from the current server and regenerated the client from it. The solve request, solve response and failure envelope were identical to the published release. Regenerating from the release's own contract reproduced the published client byte for byte, and its tests passed. The only difference from today's dump was the web framework's generic validation-error schema, which no solve call reads.
2. **The generator turns the contract into a client.** It produces typed request and response models, one method per operation, bearer authentication, typed exceptions, and the runtime policy set in the project's configuration (timeouts and retries).
3. **The output is checked, then released.** The generated models must parse payloads captured from the server application, and the client is regenerated from the committed contract and compared file by file before a release. Releases are tagged, for example `helixor-solver-contract-sdk/v0.1.0`. The same generator produces the clients for Helixor's other APIs, in Python, TypeScript and Java.

Installing it

The package is distributed from a private Helixor repository under a proprietary license. It is not on a public package index, so ask Helixor for access. Without access, use raw REST, which needs nothing private.

```
pip install "helixor-solver-contract @ git+https://github.com/HelixorAI/helixor-sdk-
artifacts.git@helixor-solver-contract-sdk/v0.1.0#subdirectory=helixor-solver/python"
```

The same solve, two ways

Both columns send the request from section 12 to the same server and read the same answer. The comments show what each one printed.

RAW REST

```
import json, os, httpx

problem = json.load(open(
    "pack/datasets/two_lane_week.demo.json"))

resp = httpx.post(
    f"{os.environ['SOLVER_URL']}"
    "/api/v1/solve/ocean_container_routing",
    headers={
        "Authorization":
            f"Bearer {os.environ['SOLVER_TOKEN']}",
        "X-Helixor-Solve-Id": "rest-demo-week-01",
    },
    json={"problem": problem,
          "solver": {"mode": "local"},
          "options": {}},
    timeout=120,
)
if resp.status_code != 200:
    raise SystemExit(f"{resp.status_code} "
                     f"{failure_code(resp.json())}")

answer = resp.json()              # a dict
sol = answer["solution"]
print(answer["status"], sol["is_feasible"],
      sol["penalty"], answer["job_id"])
# completed True 1550.0 job-…
```

GENERATED SDK

```
import json, os
from helixor_solver_contract import ApiClient, models
from helixor_solver_contract.errors import (
    ApiHttpException)

problem = json.load(open(
    "pack/datasets/two_lane_week.demo.json"))

with ApiClient(
    base_url=os.environ["SOLVER_URL"],
    bearer_token=os.environ["SOLVER_TOKEN"],
    headers={"X-Helixor-Solve-Id": "sdk-demo-week-01"},
) as client:
    try:
        answer = client.solve_api_v1_solve_problem_type_post(
            "ocean_container_routing",
            models.ApiSolveRequest(
                problem=problem,
                solver={"mode": "local"}, options={}),
        )                          # HostedSolveResponse
    except ApiHttpException as exc:
        raise SystemExit(f"{exc.raw_response.status_code} "
                         f"{failure_code(exc.parsed_error)}")

sol = answer.solution
print(answer.status, sol["is_feasible"],
      sol["penalty"], answer.job_id)
# completed True 1550.0 job-…
```

Both columns use one small helper, because failure codes sit at the top level or one level down (see When the solve fails):

```
def failure_code(body):
    if isinstance(body, dict):
        if "failure_code" in body:
            return body["failure_code"]
        if isinstance(body.get("detail"), dict):
            return body["detail"].get("failure_code")
    return None                    # a field-error list or a sentence
```

Both return the same plan, and in both `solution` is a JSON object. The SDK types the response *envelope*, but the plan the manifest engine returns inside `solution` and `result` is open, because its fields depend on the pack. So the acceptance check in Using the answer reads the same in both, and so does the failure handling: `exc.parsed_error` is the same failure body you would parse from `resp.json()`, in the same three shapes.

SDK methods and types against the REST calls

REST call	SDK method on `ApiClient`	Request type → result type
`POST /api/v1/solve/{problem_type}`	`solve_api_v1_solve_problem_type_post(problem_type, body)`	`ApiSolveRequest` → `HostedSolveResponse`
`GET /api/v1/solve/{job_id}`	`job_status_api_v1_solve_job_id_get(job_id)`	→ `dict`
`POST /api/v1/feasibility/check`	`feasibility_api_v1_feasibility_check_post(body)`	`FeasibilityCheckRequest` → `FeasibilityCheckResponse`
`POST /api/v1/feasibility/check-quick`	`feasibility_quick_api_v1_feasibility_check_quick_post(body)`	`dict` → `QuickFeasibilityResponse`
`POST /api/v1/recommendations/{problem_type}`	`recommendations_api_v1_recommendations_problem_type_post(problem_type, body)`	`RecommendationRequest` → `RecommendationsResponse`
`GET /api/v1/recommendations/{problem_type}/types`	`recommendation_types_api_v1_recommendations_problem_type_types_get(problem_type)`	→ `RecommendationTypesResponse`
`GET /api/v1/health`, `GET /api/v1/version`	`health_api_v1_health_get()`, `version_api_v1_version_get()`	→ `dict`

The pack and manifest routes (publish, versions, history, manifests, demo datasets) are not in this client: it covers the solve contract only. Call them over REST as in section 12, or use `python -m ocean_routing publish-pack`.

The two solve types, field by field:

JSON field	SDK type	Notes
`ApiSolveRequest` (the request body)		
`problem`, `solver`, `options`, `context`	`dict[str, Any] | None`	Each must be an object. `ApiSolveRequest(solver="local")` raises a validation error before anything is sent; over REST the server would refuse it.
`HostedSolveResponse` (the 200 answer)		
`status`	`str` (required)	`"completed"`. The only required field.
`job_id`, `solver_used`	`str | None`	
`objective_value`	`float | None`	0.0 for this pack: rank plans by `solution["penalty"]`.
`solution`, `result`	`dict[str, Any] | None`	The plan, certificate and pack provenance, documented in section 12.
`contract`	`SolveOutcome | None`	The typed solve-contract verdict that some built-in problem types return. Pack problem types do not, so it is `None` here.
`solve_time_ms`, `actual_provider`, `actual_device`, `solver_version`	attributes, untyped	Not in the contract, but kept: `answer.actual_provider` reads `"local"`, and `answer.model_extra` lists them all.

What the SDK adds, and what it doesn't

	Raw REST	Generated SDK
Host	You build each URL.	`base_url` is required. There is no default host, so the client never calls an address you did not configure.
Auth	You add `Authorization: Bearer …` to every call.	`bearer_token=`, either a string or a function returning a fresh token, is sent on every call. Other headers, such as `X-Helixor-Solve-Id`, go in `headers=`.
Request shape	Checked by the server: a wrong type comes back as a 422.	Field types are checked when you build `ApiSolveRequest`, before sending. Unknown extra fields are passed through, not refused, because this client is configured to allow them.
Response shape	Whatever JSON arrives.	Validated against `HostedSolveResponse`. A field a later server release adds is kept, never a parse failure: responses are tolerant.
Failures	Check `status_code`, then parse the body.	A non-2xx answer raises `ApiHttpException`, with `raw_response.status_code` and the failure body in `parsed_error` (a `dict`). A response that breaks the contract raises `ApiResponseValidationException`. A broken connection still raises the HTTP library's own error, as it would over REST.
Retries and timeouts	Yours to write.	120 s default timeout (pass `timeout=` to change it). Up to 2 attempts on 408, 429, 502, 503 and 504, for GET, HEAD and OPTIONS only. A solve (POST) is never retried, so it never runs twice behind your back.
Contract record	None.	`OPERATION_CONTRACTS` lists every operation with a `sha256` of its wire shape, so you can tell exactly which contract the client was built from.
Plan contents	The same in both: an open JSON object you read with the fields in section 12.	

Choose the SDK when you want the envelope, authentication, failure handling and retries settled for you, and want each upgrade to be a new, versioned client. Choose raw REST when you cannot take a private dependency, or need the pack routes the SDK leaves out. This repository uses raw REST for both reasons.

14. Running the solver locally: the embedded runtime

Everything above calls a Solver service over the network. The **Helixor Decision Runtime** (the proprietary `helixor-runtime` package; ask Helixor for access) can instead run Helixor's solver inside your own

process. In that mode there is no network call, the records never leave your host, and the answer comes back as a function result. This section shows how to switch it on, what it needs, and what it can run today. Every result here comes from runtime code current on 2026-11-08, run on this example's demo week.

One explicit switch

```
from pii_guard.license import LicenseVerifier
from helixor_runtime import HelixorSolver

# Embedded: helixor-solvers runs in this process and never calls a service.
license = LicenseVerifier.load("helixor.hxlic")         # verifies the signature
solver = HelixorSolver(mode="embedded", license=license)

# Hosted: the same calls, delegated to a Solver service through its generated client.
solver = HelixorSolver(mode="hosted", base_url=SOLVER_URL, auth=SOLVER_TOKEN)
```

The mode has no default, and neither mode falls back to the other. `HelixorSolver(mode="")` raises `SolverConfigurationError`. A missing engine or an unreachable service is also a typed error, never a quiet switch to the other mode. Both modes expose `solve`, `feasibility` and `recommendations`, return the same solve contract (`verdict`, the plan only when it is feasible, named `shortfalls`, ranked `alternates`, a `certificate`, and `metadata.model` saying which mode answered), and raise the same typed errors. Install the `solvers` extra for routing; rostering needs `solvers-full`.

The license decides

The embedded mode runs only under a signed Helixor license whose entitlements cover the solver. The runtime checks it on every solve:

License	Embedded routing solve
none	`SolverLicenseRequiredError`: the solver "requires a Helixor license; none was supplied"
Developer tier	`LicenseEntitlementError`: not authorized, because its licensed solvers cover only `solver.tcn.*`
Enterprise tier	solved: `verdict: "feasible"`, `metadata.model: "embedded"`

The hosted mode takes no license: the service authorizes each call by your token, as in section 12.

What it can run today, and what it can't

The runtime's solve contract covers two problem types: `vrp` (vehicle routing) and `rostering`. This example's problem type is a **solver pack**, and the embedded mode does not run solver packs yet. It refuses the type, by name, rather than answering something else:

```
problem = json.load(open("pack/datasets/two_lane_week.demo.json"))
try:
    solver.solve("ocean_container_routing", problem)
except SolverOptionError as exc:
    print(exc.failure_code, "|", exc)
# unsupported_problem_type | no solve contract for problem type
#   'ocean_container_routing'; supported: ['rostering', 'vrp']
```

The runtime's hosted mode can't serve this pack either. It expects a solve-contract answer, and a pack solve returns the plan and certificate documented in section 12 instead. It raises `SolverServiceError` ("answered … without a solve-contract answer"). For this problem, call the Solver API directly, over raw REST or the generated client (section 13).

Running this example on your own machine

To run the container routing pack locally today, point `SOLVER_URL` at a Solver server running on your own host instead of the hosted one. Then publish the pack to it (`python -m ocean_routing publish-pack`), and every command and example in sections 12 and 13 runs unchanged. That is how this guide's examples were produced: the server ran on the same machine as the client, and the demo week solved in about 30 ms once warm. The first solve on a freshly started server took about 45 seconds. The Solver server is not part of this repository.

	Hosted service	Solver server on your host	Embedded runtime
Where it runs	Helixor's service	a separate process on your machine	inside your process
Called by	REST or the generated client	REST or the generated client	a function call
Authorized by	your bearer token	the server's own sign-in settings	your signed license
This pack (`ocean_container_routing`)	yes, once the pack is published	yes, once the pack is published (verified)	**not yet**: `unsupported_problem_type`
`vrp` , `rostering`	served (not run for this guide)	served (not run for this guide)	yes, with an entitled license (routing verified)

Not yet available. Running a solver pack in-process through `HelixorSolver(mode="embedded")` is not supported. Until it is, use the hosted service or a Solver server on your own host, and treat any other claim of an embedded pack solve as unverified.

15. Run it yourself

Hosted solves need a Helixor account at helixor.dev whose organization holds a **Helixor Solver license**, plus the Solver API origin and a token for your account. The repository itself contains no solver. Everything marked offline runs without a license.

```
git clone https://github.com/HelixorAI/helixor-solver-ocean-container-routing.git
cd helixor-solver-ocean-container-routing
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pytest -q                                  # offline tests; live tests run only with
SOLVER_LIVE=1
```

Step	Command	Needs	About
Proven optimum	`python -m ocean_routing optimum`	offline	1 s
Score any plan	`python -m ocean_routing score out/plan-optimum.json`	offline	1 s
Replay, MIP repairs	`python -m ocean_routing continuous --start out/plan-optimum.json --offline` writes out/continuous-offline/	offline	5 s
Report	`python -m ocean_routing report out/plan-optimum.json -o out/report.html`	offline	1 s
Publish the pack	`python -m ocean_routing publish-pack`	license	5 s
First plan	`python -m ocean_routing plan`	license	1 min
Adopt a plan	`python -m ocean_routing adopt out/plan-optimum.json`	license	5 s
Hosted replay + comparisons	`python -m ocean_routing continuous --start out/plan-adopted.json --compare`	license	10 min
Pinning scenarios	`python -m ocean_routing pinning`	license	7 min

For hosted steps, copy `.env.example` to `.env`, fill in `SOLVER_URL` and `SOLVER_TOKEN`, and load it with `set -a && . ./.env && set +a`. Outputs land in `out/`. The README has the full command reference and every measured table; the tutorial explains each rule and file in depth. Source: https://github.com/HelixorAI/helixor-solver-ocean-container-routing.

16. Glossary

FEU	Forty-foot equivalent unit. A 40' container is 1 FEU, a 20' container 0.5.
Lane	An origin–destination port pair, such as CNSHA-USLAX.
Sailing	One weekly voyage of one carrier service, for example ATLS-PSX-W03: Atlas Line, service PSX, week 3.
Cutoff	The last day the container yard accepts cargo for a sailing.
Allocation	Weekly space a carrier has contracted to the importer on a sailing. Already acknowledged.
Spot	Space beyond the allocation, bought at market rates, up to what the carrier has on sale.
Promise date	The day the container's customer was promised delivery.
Solver pack	A versioned zip of configuration (manifest, rules, weights, a demo dataset) published to the hosted solver. Versions are immutable.
Lowering	Turning your records into the request: deriving on-time sailings, capacity, confirmed and committed space.
Certificate	The solver's account of a plan: feasible or not, and each rule's penalty.
Pin	Per-container instruction in a repair: `frozen`, `forbid`, `prefer`, or none.
Change cost	The price of moving a `prefer` container, in the same units as the weights.
Confirmed / committed	Confirmed: space the carrier acknowledged. Committed: confirmed plus pending requests.